

Event-Driven Causal Runtime-Adaptive CPU Scheduling with Sequential Tabular Q-Learning

Single-CPU Discrete-Event Simulation, Causal Arrived-Work State Abstraction, Undiscounted Waiting-Time Learning, Predeclared Heuristic Comparison, and Held-Out Crossed-Bootstrap Evaluation

6 Methods	162 × 4 = 648	6,000 Traces	140 / 210	273 / 273
4 Fixed + Heuristic + Runtime Q	Causal States × Actions	5 Seeds × 1,200 Train Episodes	Val / Held-Out Test Workloads	Automated Pytest Suite Passing

1. Review 2 Milestone: From Offline Selector to Causal Runtime Adaptation

In **Review 1**, the repository established four verified standalone CPU schedulers—**First-Come First-Served (FCFS)**, **non-preemptive Shortest Job First (SJF)**, **Round Robin (RR)**, and **non-preemptive Priority Scheduling**—alongside an initial whole-workload offline policy selector that inspected complete pre-execution workload summaries and counterfactual four-policy schedules. For **Review 2**, the project delivers a true **single-CPU, discrete-event, causal runtime-adaptive scheduler** (`scheduler/runtime.py`, `rl/runtime_controller.py`, `experiments/runtime_experiment.py`). At every dispatch or Round-Robin quantum boundary, the online controller observes only arrived work and selects FCFS, SJF, Round Robin, or Priority for the next execution segment of the same evolving process trace.

Design Dimension	Legacy Offline Selector (Review 1)	Causal Runtime Scheduler (Review 2 Core)
Decision Granularity	One pre-execution batch action per workload.	Sequential choice at every dispatch or RR quantum boundary in one trace.
Information Boundary	Receives full Workload including future arrivals.	Immutable RuntimeObservation of arrived/ready jobs only.
Ready-Queue State	Runs one fixed standalone policy from $t=0$ to completion.	Persistent FIFO ready queue; preserves remaining bursts & insertion order.
Tabular State Space	4 whole-workload features × 3 bins = 81 states.	5 causal arrived-work features ($3 \times 3 \times 3 \times 3 \times 2$) = 162 states (648 pairs).
Reward Signal	Terminal counterfactual deviation from 4-policy mean.	Interval waiting reward $r_t = -\Delta W_t / \max(1, q)$, $\gamma = 1.0$ ($\Sigma r_t = -\text{Wait}/q$).
Switch Cost Model	Switching cost = 0 (counted only in reward).	Quantum $q = 4$, switch cost = 1 unit per running-PID change for all methods.
Unseen-State Handling	Zero-initialized Q-tie defaults to index 0 (FCFS).	First-visit random action; unvisited masking; explicit heuristic fallback.

Table 1.1: Architectural progression from the legacy offline selector to the Review 2 causal runtime scheduler.

2. Event-Driven Execution Semantics & Ready-Queue Contract

Causal Observation Boundary (RuntimeObservation): The environment privately owns the future arrival list. At decision epoch t , the controller receives only: (1) current simulated time and immediately preceding policy (None on first call); (2) currently ready processes that have already arrived (PID, arrival time, known burst, remaining burst, priority); (3) counts of arrived and completed processes; (4) mean burst and priority spread computed strictly over arrived processes so far; and (5) mean arrival age (`current_time - arrival_time`) of currently ready jobs. Unarrived process attributes, total process counts, and counterfactual outcomes never cross the controller API. Exact burst length becomes visible upon arrival (matching standalone SJF).

Policy / Mechanism	Event-Driven Runtime Execution & Ready-Queue Rule
Persistent Ready Queue	Single FIFO of arrived, unfinished, non-running jobs. Simultaneous arrivals enter by canonical (<code>arrival_time</code> , <code>pid</code>) order. Dispatch removes the chosen job; completion removes it permanently.
FCFS (Action 0)	Dispatches current ready-queue head non-preemptively to completion (follows live FIFO insertion order).
SJF (Action 1)	Dispatches ready job with smallest <code>remaining_burst</code> non-preemptively (ties preserve current queue order).
Round Robin (Action 2)	Runs queue head for $\min(q=4, \text{remaining_burst})$; endpoint arrivals enter in (<code>arrival_time</code> , <code>pid</code>) order before unfinished job re-enqueues at tail.
Priority (Action 3)	Dispatches ready job with highest static priority non-preemptively (lower number = higher priority; ties keep queue order).
Context-Switch Cost	Cost = 1 unit charged only when running PID changes (initial dispatch & same-PID continuation free; switch-interval arrivals do not revoke dispatch).

Table 2.1: Deterministic single-CPU runtime execution semantics and ready-queue contract.

3. Causal State Abstraction, Sequential Q-Learning & Heuristic Baseline

Mixed-Radix Causal State Space (162 States): RuntimeStateEncoder accepts only a RuntimeObservation and discretizes five causal features into $3 \times 3 \times 3 \times 3 \times 2 = 162$ tabular states (648 state-action pairs across the 4 scheduling actions):

Feature Index & Name	Radix	Discretization Bins (q = 4)	Causal Rationale & Scope
1. ready_count	3	{1}, {2, 3}, {4+}	Measures instantaneous ready-queue contention.
2. completed_count	3	{0}, {1-3}, {4+}	Tracks trace progression without knowing total future jobs.
3. median_ready_remaining_burst	3	$\leq q (\leq 4)$, $\leq 4q (5-16)$, $> 4q (>16)$	Captures remaining service demand of currently ready jobs.
4. mean_ready_arrival_age	3	$\leq q (\leq 4)$, $\leq 4q (5-16)$, $> 4q (>16)$	Mean (current_time – arrival_time) of ready jobs (age proxy).
5. arrived_work_priority_spread	2	Zero (0), Non-zero (>0)	Indicates whether arrived processes have distinct priorities.

Table 3.1: Five-feature causal state encoding in rl/runtime_state.py ($3 \times 3 \times 3 \times 3 \times 2 = 162$ states).

Sequential Q-Learning & Undiscounted Waiting-Cost Objective: Each runtime trace is one sequential episode. Between decision epochs t and $t+1$, let ΔW_t be the actual waiting-time area accumulated by ready processes (including any 1-unit PID-switch interval). The step reward is $r_t = -\Delta W_t / \max(1, q)$. With discount factor $\gamma = 1.0$, the episodic return over a completed finite workload satisfies $\sum_t r_t = -\text{TotalWaitingTime} / q$ exactly. Non-terminal transitions update via $Q(s,a) \leftarrow Q(s,a) + \alpha [r_t + \gamma \max_{a' \in \text{visited}(s)} Q(s',a') - Q(s,a)]$ ($\alpha = 0.1$), while terminal transitions omit the bootstrap. The first visit to any state forces a uniform random action; greedy selection and bootstrapping mask unvisited actions; wholly unseen evaluation states invoke the explicit unseen-state-causal-fallback (causal heuristic).

Predeclared Non-RL Causal Heuristic (rl/runtime_heuristic.py): To evaluate whether sequential RL adds value beyond hand-crafted rules under the exact same observation contract, a fixed deterministic rule is evaluated alongside Q-learning: (1) select **Round Robin** if ≥ 3 jobs are ready and their mean arrival age $\geq q$; (2) else select **Priority** if ≥ 2 ready jobs have distinct priorities; (3) else select **SJF** if $\max(\text{remaining_burst}) \geq 2 \times \min(\text{remaining_burst})$ among ready jobs; (4) else **FCFS**.

4. Experimental Design, Workload Families & Three-Way Split Integrity

The runtime study uses **seven synthetic workload families** (15 processes per workload) with SHA-256 derived seeds (derive_seed). Six families cycle during training (1,200 sequential episodes per seed across 5 independent agent seeds $7101-7105 = 6,000$ training workloads), while poisson_arrivals is held out entirely from training to test out-of-family arrival generalization. Validation uses master seed 8201 (20 reps $\times$ 7 families = 140 workloads) for reporting and predeclared demo selection only. Final testing uses untouched master seed 19301 (30 reps $\times$ 7 families = 210 workloads; prior audited seed 9301 was retired after methodology remediation). Train/validation/test SHA-256 workload fingerprint overlap is verified to be strictly False.

Workload Family	Split Role	Burst Distribution (Time Units)	Arrival Pattern	Priority Pattern
short_jobs	Train / Val / Test	Bimodal: 80% in [1, 5], 20% in [6, 50]	Uniform in [0, 20]	Uniform in [1, 5]
long_jobs	Train / Val / Test	Bimodal: 20% in [1, 5], 80% in [6, 50]	Uniform in [0, 20]	Uniform in [1, 5]
mixed	Train / Val / Test	Uniform over full range [1, 50]	Uniform in [0, 20]	Uniform in [1, 5]
cpu_bursty	Train / Val / Test	Long uniform bursts in [20, 50]	Dense uniform in [0, 15]	Uniform in [1, 5]
priority_skewed	Train / Val / Test	Uniform over [1, 50]	Uniform in [0, 20]	70% high priority [1, 2]
staggered_interactive	Train / Val / Test	1 long [25, 50] at $t=0$ + 14 short [1, 3]	Staggered gaps 4–6 units	All equal (priority = 1)
poisson_arrivals	Held-Out (Val/Test)	Uniform in [1, 20]	Poisson process ($\lambda = 0.5$)	Uniform in [1, 5]

Table 4.1: Seven declared synthetic workload families (15 processes/workload). poisson_arrivals is held out from training.

Split Audit & Cryptographic Fingerprint Verification (results/runtime/runtime_summary.json):

- Training Union SHA-256 (6,000 workloads):**
e974835c99b6247c0d84b43d9b565a0b99bc28db6ac3e3d5ee2bb0f956e098be
- Validation Split SHA-256 (140 workloads):**
d5925901863d4063b8df7083837aacc5ddf77e73813362a870c918784d7d33a2
- Held-Out Final Test SHA-256 (210 workloads):**
9fa14bbae79d39c470174fdc2e99a467e171677f146bc755ae11ad89a8f82554

5. Measured Runtime-Adaptation Results (Held-Out Final Test & Validation)

All six methods are evaluated on the exact same 210 held-out final-test workloads (master seed 19301) and 140 validation workloads (master seed 8201) with quantum $q = 4$ and context-switch cost = 1 time unit. Fixed policies and the causal heuristic produce 1 deterministic run per workload (210 rows); Runtime Q-learning evaluates all 5 independently trained agents greedily in read-only mode (1,050 rows, averaged equally across seeds).

Method (Final Test, Seed 19301)	Mean Wait $\pm$ SD	Turnaround $\pm$ SD	Response $\pm$ SD	Util %	Thru.	Ctx Switches	Pol. Sw.
FCFS	127.797 $\pm$ 79.81	146.612 $\pm$ 90.51	127.797 $\pm$ 79.81	92.237%	0.077	14.000 $\pm$ 0.00	0.000
SJF (Non-Preemptive)	93.345 $\pm$ 68.78	112.161 $\pm$ 79.39	93.345 $\pm$ 68.78	92.237%	0.077	14.000 $\pm$ 0.00	0.000
Round Robin ($q = 4$)	215.729 $\pm$ 172.82	234.544 $\pm$ 183.45	26.220 $\pm$ 9.15	78.216%	0.068	74.767 $\pm$ 40.70	0.000
Priority (Non-Preemptive)	129.207 $\pm$ 80.81	148.022 $\pm$ 91.53	129.207 $\pm$ 80.81	92.237%	0.077	14.000 $\pm$ 0.00	0.000
Causal Heuristic (Predeclared)	203.404 $\pm$ 159.23	222.220 $\pm$ 169.85	40.070 $\pm$ 17.05	80.481%	0.071	67.371 $\pm$ 39.49	3.210
Runtime Q-Learning (5 Seeds)	104.789 $\pm$ 71.43	123.604 $\pm$ 81.86	86.341 $\pm$ 59.82	90.836%	0.076	17.322 $\pm$ 4.66	4.729

Table 5.1: Held-out final test results across 210 unique workloads (master seed 19301). Validation split (140 workloads, seed 8201) shows closely matching means: Runtime Q-learning wait = 104.896, SJF wait = 95.797, Heuristic wait = 209.366.

Causal Runtime-Adaptive Experiment: Final Held-Out Test Performance (210 Workloads, Master Seed 19301)

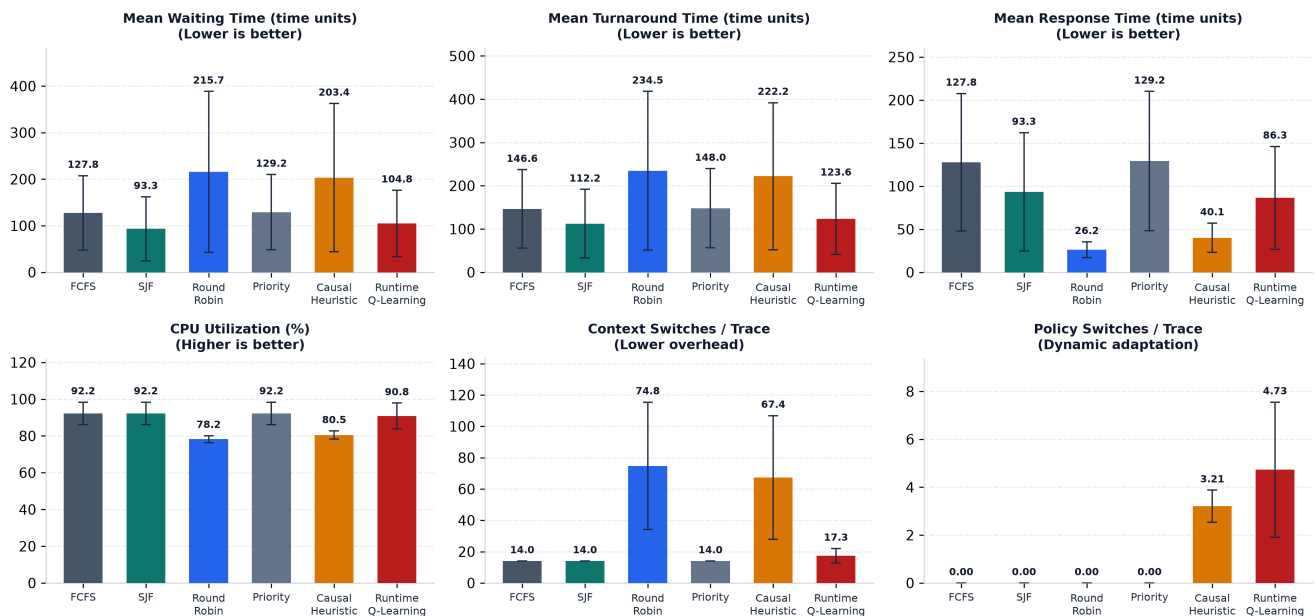


Figure 5.1: Six-metric comparison on the 210 held-out final-test workloads (mean $\pm$ SD across workloads and 5 model seeds).

Key Empirical Findings on the Held-Out Final Test:

- **Comparison vs. Fixed Baselines:** Runtime Q-learning substantially reduces mean waiting time compared to Round Robin (**104.789 vs. 215.729**, -51.4%), Priority (**129.207**, -18.9%), and FCFS (**127.797**, -18.0%), while improving mean response time over SJF (**86.341 vs. 93.345**, paired difference -7.004 , 95% CI $[-9.541, -4.305]$).
- **Honest Reporting vs. SJF on Waiting Time:** Because SJF is non-preemptive and receives exact burst lengths upon arrival without incurring mid-job RR preemption/switch overhead, fixed SJF achieves the lowest overall mean waiting time (**93.345**). Runtime Q-learning does not beat SJF on aggregate waiting time ($Q - SJF = +11.443$, 95% CI $[+5.022, +19.297]$), though Q-learning outperforms SJF on staggered_interactive workloads (**10.905 vs. 17.062** mean wait; **9.152 vs. 17.062** response time).
- **Comparison vs. Predeclared Causal Heuristic:** Runtime Q-learning beats the causal heuristic by -98.616 time units on mean waiting time (95% CI $[-106.097, -90.192]$) and reduces context switches by -50.050 per trace, whereas the heuristic favors frequent Round Robin slices and therefore achieves lower response time (40.070 vs. 86.341).

6. Workload-Family Breakdown & Sequential Learning Dynamics

Inspecting performance across the seven individual workload families reveals how the sequential Q-learning controller adapts its policy selections to ready-queue conditions, including on the held-out poisson_arrivals distribution:

Workload Family (n=30)	FCFS Wait / Resp	SJF Wait / Resp	RR Wait / Resp	Priority Wait / Resp	Heuristic Wait / Resp	Runtime Q Wait / Resp (Sw.)
short_jobs	56.85 / 56.85	22.88 / 22.88	47.52 / 20.90	56.22 / 56.22	46.24 / 25.76	40.47 / 35.59 (6.12)
long_jobs	151.75 / 151.75	97.81 / 97.81	228.69 / 30.02	154.90 / 154.90	215.50 / 46.71	114.18 / 96.76 (3.64)
mixed	171.65 / 171.65	121.23 / 121.23	283.69 / 32.20	175.89 / 175.89	266.52 / 51.00	133.07 / 109.70 (3.03)
cpu_bursty	244.49 / 244.49	213.51 / 213.51	518.70 / 33.22	246.21 / 246.21	483.24 / 58.79	223.74 / 184.01 (2.85)
priority_skewed	183.06 / 183.06	131.16 / 131.16	308.26 / 31.76	184.45 / 184.45	287.98 / 48.61	142.25 / 118.60 (3.00)
staggered_interactive	18.63 / 18.63	17.06 / 17.06	9.65 / 6.66	18.63 / 18.63	18.59 / 18.59	10.91 / 9.15 (8.84)
poisson_arrivals (Held-Out)	68.14 / 68.14	49.76 / 49.76	113.58 / 28.78	68.15 / 68.15	105.77 / 31.03	68.91 / 50.59 (5.63)

Table 6.1: Per-family mean waiting time / mean response time (and mean policy switches for Runtime Q-learning) across all 7 families.

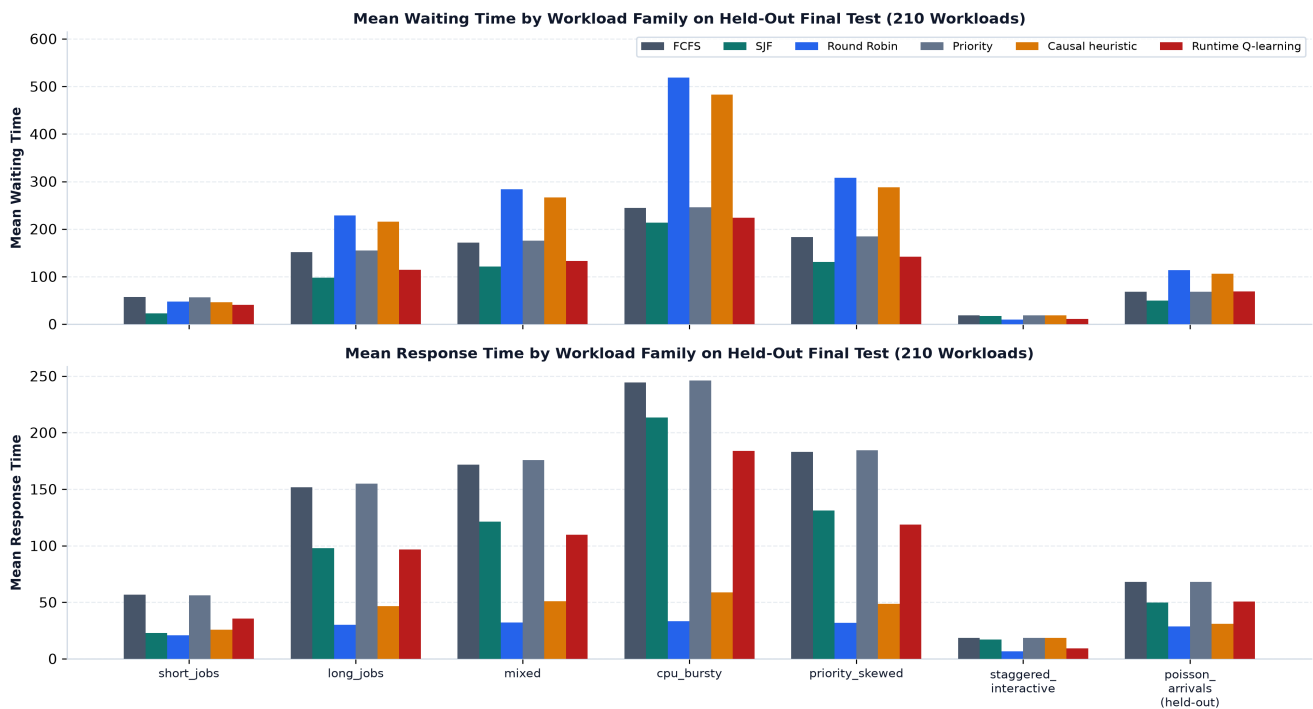


Figure 6.1: Held-out final test mean waiting time (top) and mean response time (bottom) across the 7 workload families.

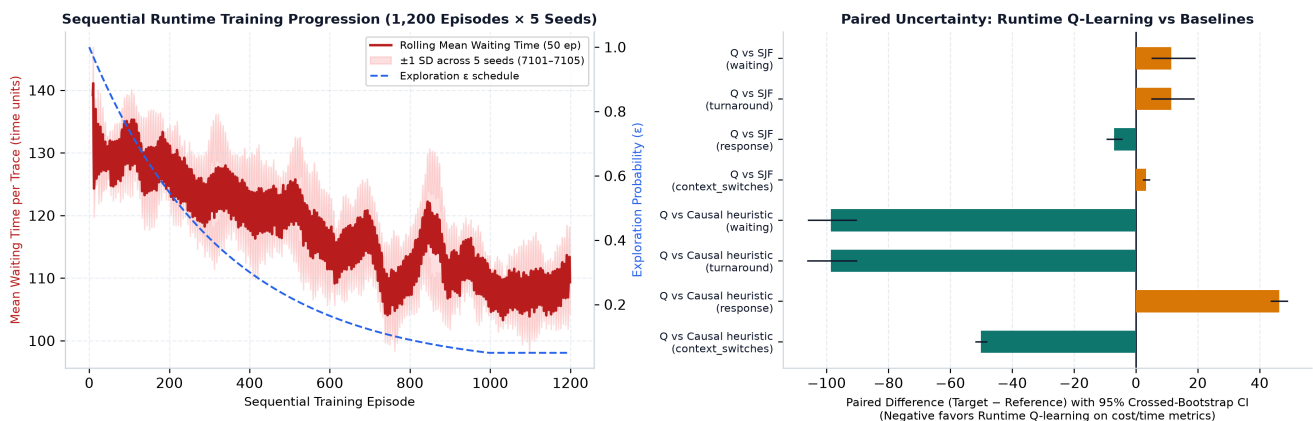


Figure 6.2: Left: Sequential training waiting-time reduction across 1,200 episodes (5 seeds). Right: Paired 95% crossed-bootstrap CIs.

7. Paired Uncertainty Table, State Coverage & Illustrative Trace Analysis

Family-Stratified Crossed Bootstrap (1,000 Replicates, Seed 104729): Paired differences (Target – Reference) resample test workloads within each family and independently resample the 5 trained model seeds:

Target Method	Reference	Metric	Mean Diff.	95% Crossed-Bootstrap CI	Empirical Interpretation
Runtime Q-learning	SJF	avg_waiting_time	+11.443	[+5.022, +19.297]	SJF lower wait (no RR switch cost)
Runtime Q-learning	SJF	avg_response_time	−7.004	[−9.541, −4.305]	Favors Runtime Q-learning
Runtime Q-learning	SJF	cpu_utilization (%)	−1.402	[−1.843, −1.005]	Reflects +3.322 switch overhead
Runtime Q-learning	SJF	context_switches	+3.322	[+2.209, +4.555]	Modest switching vs RR (+60.767)
Runtime Q-learning	Causal heuristic	avg_waiting_time	−98.616	[−106.097, −90.192]	Strongly favors Runtime Q-learning
Runtime Q-learning	Causal heuristic	avg_response_time	+46.272	[+43.472, +49.100]	Heuristic RR slices lower response
Runtime Q-learning	Causal heuristic	cpu_utilization (%)	+10.354	[+9.890, +10.834]	Favors Runtime Q-learning
Runtime Q-learning	Causal heuristic	context_switches	−50.050	[−51.929, −48.066]	50 fewer switches/trace than heuristic

Table 7.1: Paired differences and 95% family-stratified crossed-bootstrap confidence intervals on the final test set.

Training Seed	Train Decisions	Visited States (of 162)	Visited (s, a) Pairs (of 648)	Train Wall / Q-Update (ms)	Val Decisions / Unseen Fallbacks	Test Decisions / Unseen Fallbacks
7101	23,511	81 (50.0%)	267 (41.2%)	2022.0 / 424.0 ms	2,697 / 3 (0.11%)	4,153 / 2 (0.05%)
7102	23,398	89 (54.9%)	272 (42.0%)	2050.2 / 426.4 ms	2,812 / 1 (0.04%)	4,262 / 1 (0.02%)
7103	23,596	87 (53.7%)	273 (42.1%)	1977.2 / 416.9 ms	2,486 / 1 (0.04%)	3,769 / 1 (0.03%)
7104	23,358	83 (51.2%)	276 (42.6%)	2044.3 / 430.6 ms	2,325 / 2 (0.09%)	3,532 / 3 (0.08%)
7105	23,671	88 (54.3%)	275 (42.4%)	2042.0 / 432.4 ms	2,468 / 3 (0.12%)	3,738 / 1 (0.03%)

Table 7.2: Per-seed tabular state-action coverage and unseen-state fallback counts. Mean decision overhead: 13.693 μ s (observation) + 21.239 μ s (selection) = 34.932 μ s/decision (0.647 ms/trace).

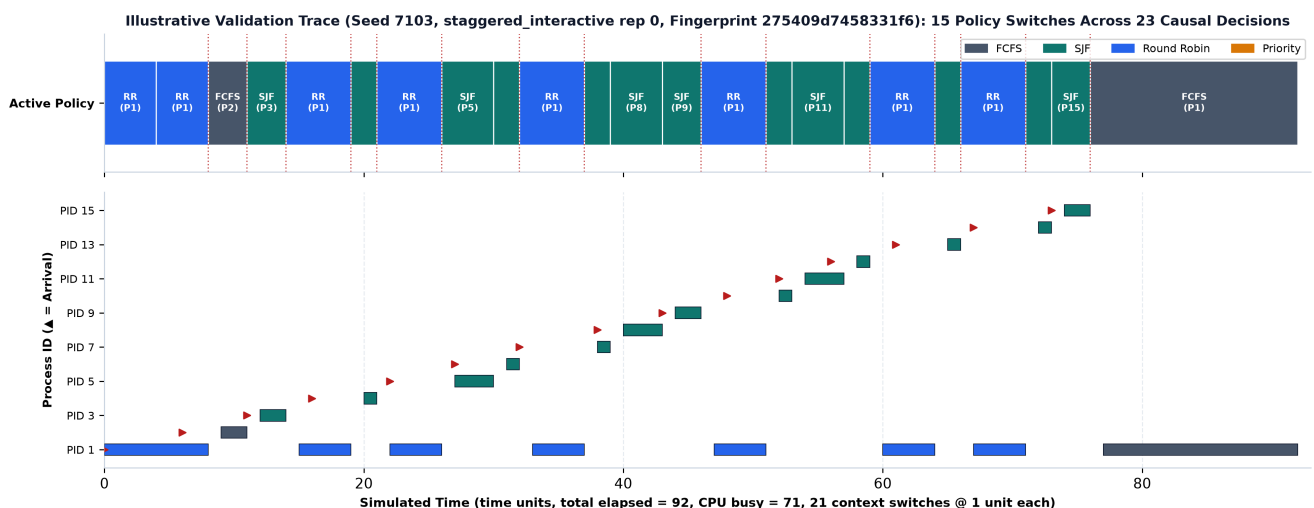


Figure 7.1: Illustrative validation-split trace (Seed 7103, staggered_interactive rep 0, fingerprint 275409d7458331f6). Selected by predeclared validation-only rule (15 policy switches over 23 decisions, 0 fallbacks; wait = 5.933, response = 2.933). Explicitly illustrative, not representative of average behavior.

8. Preserved Legacy Offline Policy Selector Reference & Visual Artifacts

For full repository continuity, the earlier whole-workload offline policy selector (python main.py experiment) remains intact and verified under results/ and figures/. Because it observes complete-workload pre-execution features (81 states) and trains with counterfactual four-policy rewards (switching cost = 0), it is documented separately and never mixed into causal runtime claims:

Offline Evaluation Policy (210 Workloads, Cost = 0)	Mean Wait $\pm$ SD	Turnaround $\pm$ SD	Response $\pm$ SD	CPU Util %	Ctx Switches	Wait Reduction vs Baseline
FCFS	123.39 $\pm$ 80.71	142.34 $\pm$ 91.49	123.39 $\pm$ 80.71	97.84 $\pm$ 6.19%	14.00 $\pm$ 0.00	+28.1% (Selector lower)
SJF	88.10 $\pm$ 67.11	107.06 $\pm$ 77.80	88.10 $\pm$ 67.11	97.84 $\pm$ 6.19%	14.00 $\pm$ 0.00	-0.6% (Near-oracle tie)
Round Robin (q = 4)	167.57 $\pm$ 135.06	186.52 $\pm$ 145.75	19.38 $\pm$ 8.22	97.84 $\pm$ 6.19%	75.40 $\pm$ 40.91	+47.1% (Selector lower)
Priority	122.22 $\pm$ 81.44	141.17 $\pm$ 92.25	122.22 $\pm$ 81.44	97.84 $\pm$ 6.19%	14.00 $\pm$ 0.00	+27.5% (Selector lower)
Offline Policy Selector (Q-Learning, 5 Seeds)	88.66 $\pm$ 67.89	107.61 $\pm$ 78.54	88.11 $\pm$ 68.47	97.84 $\pm$ 6.17%	15.53 $\pm$ 4.04	Selects RR on interactive, SJF on batch

Table 8.1: Historical offline whole-workload policy selector summary (seeds 42-46, 210 workloads, switching_cost = 0).

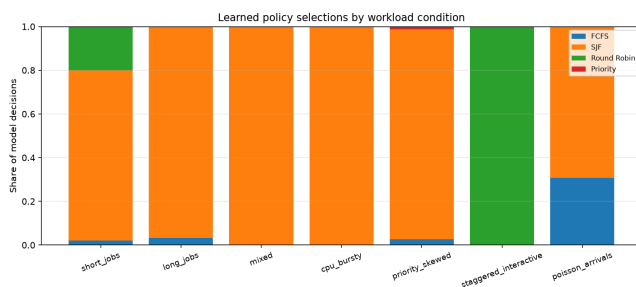


Figure 8.1: Offline selector policy share by workload condition.

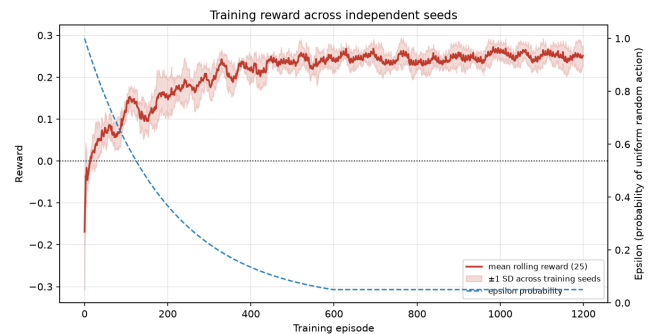


Figure 8.2: Offline selector rolling training reward & ϵ decay.

9. Verification Suite (273 Tests), Limitations & Reproducibility Commands

Comprehensive Automated Verification (273 Passing Pytest Checks):

- **Fixed-Policy Equivalence** (test_runtime_scheduler.py): Verifies that running the event-driven runtime simulator with a constant action reproduces standalone FCFS, SJF, Round Robin, and Priority schedules identically across hand-computed and seeded random traces.
- **Causal Hidden-Future Invariance**: Alters arrival times, bursts, and priorities of unarrived future processes while keeping arrived history identical; verifies that observation and action prefixes remain bit-for-bit identical.
- **Reward-to-Waiting Identity & Bootstrapping**: Confirms $\sum_t r_t == -\text{TotalWaitingTime} / q$ for $q = 1.0$, terminal bootstrap omission, first-visit exploration forcing, unvisited-action masking, and read-only evaluation.
- **Split Disjointness & Reproducibility** (test_runtime_experiment.py): Verifies zero SHA-256 workload fingerprint overlap across train/validation/test and deterministic metric regeneration.

Scope & Simulator Limitations: The simulator models a synthetic single-CPU ready queue without I/O blocking, multicore cache/bus contention, priority aging, or kernel scheduling latency. Exact burst times become known upon process arrival (matching standalone SJF). The 162-state discretization is coarse (~42% state-action coverage), so conclusions are scoped strictly to the declared workload families, quantum/switch-cost parameters, and waiting-time objective.

```
python3.11 -m venv .venv && source .venv/bin/activate && pip install -r requirements.lock
python -m pytest -q # Runs all 273 unit & integration tests
python main.py runtime-experiment # Reproduces causal runtime experiment in results/runtime/
python main.py runtime-experiment --results-dir /tmp/os-project-runtime # Isolated deterministic verification run
python main.py experiment # Reproduces legacy offline selector in results/ & figures/
```